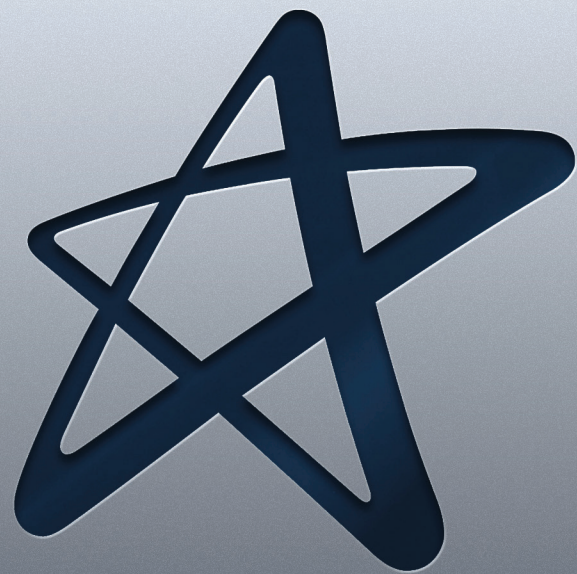


# **Computação Paralela e Distribuída**



**Cruzeiro do Sul Virtual**  
Educação a distância



# Material Teórico



**Concorrência, Replicação e *Design* em Sistemas Distribuídos**

**Responsável pelo Conteúdo:**

Prof. Esp. Allan Piter Pressi

**Revisão Textual:**

Prof.<sup>a</sup> Dr.<sup>a</sup> Selma Aparecida Cesarin



# UNIDADE

## Concorrência, Replicação e *Design* em Sistemas Distribuídos



- Transações e Controle de Concorrência;
- Transações Distribuídas;
- Replicação.



### OBJETIVO DE APRENDIZADO

- Compreender como os Sistemas Distribuídos gerenciam a Computação, os conceitos de Replicação e Desenho e a Arquitetura de Sistemas.







# Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



## Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

# Transações e Controle de Concorrência

O objetivo deste conteúdo é o conhecimento da aplicação de transações e controle de concorrência para compartilhamento de objetos gerenciados por servidores.

Uma transação pode ser definida como uma sequência de operações do servidor. As transações são aninhadas e estruturadas a partir de conjuntos de outras transações e são particularmente úteis em Sistemas Distribuídos porque permitem concorrência adicional.

Todos os Protocolos de Controle de Concorrência são baseados no critério de equivalência e são derivados de regras para a resolução de conflitos entre operações.

São descritos três métodos:

- Os bloqueios são usados para ordenar transações que acessam os mesmos objetos, de acordo com a ordem de chegada de suas operações;
- O controle de concorrência permite que as transações prossigam até que estejam prontas para serem gravadas, o que é feito após uma verificação de que operações conflitantes não tenham ocorrido;
- A ordenação de *timestamp* usa registros de data e hora para ordenar transações que acessam o mesmo objeto, de acordo com seus horários de início.

## Introdução

---

O objetivo das transações é garantir que todos os objetos gerenciados por um servidor permaneçam num estado consistente quando são acessados por várias transações e na presença de falhas no servidor.

As transações lidam com falhas de Processos e falhas de omissão em Comunicação, mas não em qualquer tipo de comportamento arbitrário.

Objetos que podem ser recuperados após suas falhas no servidor são chamados objetos recuperáveis. Em geral, os objetos gerenciados por um servidor podem ser armazenados em memória volátil (memória **RAM**) ou na memória persistente (disco rígido).

Mesmo se os objetos forem armazenados na memória volátil, o servidor pode usar a memória persistente para armazenar informações para o estado dos objetos a serem recuperados se o Processo do servidor falhar, o que permite que os servidores tornem os objetos recuperáveis.

Uma transação é especificada por um cliente como um conjunto de operações em objetos a serem executados como uma unidade indivisível pelos servidores que gerenciam esses objetos.



Os servidores devem garantir que toda a transação seja realizada e os resultados registrados em armazenamento permanente ou, no caso de um ou deles cair, que seus efeitos sejam completamente apagados.

Este item se concentra nas questões de uma transação num único servidor; no próximo item, serão discutidas questões relacionadas a transações que envolvem vários servidores e, em particular, como eles decidem sobre o resultado de uma transação distribuída.

A transação de um cliente também é considerada indivisível sob o ponto de vista das transações de outros clientes, no sentido de que as operações de uma transação não podem observar os efeitos parciais das operações de outra.

Nas Unidades anteriores, vimos que o uso de diversos *threads* é benéfico para o desempenho em muitos servidores. Notamos, também, que o uso de *threads* permite que operações de vários clientes sejam executadas simultaneamente e que seja possível acessar os mesmos objetos; portanto, os métodos dos objetos devem ser projetados para uso num contexto *multi-threaded*. Por exemplo, se os métodos para depositar e retirar não são projetados para uso num programa *multi-threaded*, então, é possível que as ações de duas ou mais execuções concorrentes do método possam ser intercaladas arbitrariamente e ter efeitos estranhos nas variáveis de instância dos objetos da conta.

Se um *thread* invoca um método sincronizado num objeto, então, esse objeto é efetivamente bloqueado e outro *thread* que invoca um dos seus métodos sincronizados será bloqueado, até que o bloqueio seja liberado.

Essa forma de sincronização força que a execução de *threads* seja separada no tempo e garante que as variáveis de instância de um único objeto sejam acessadas de maneira consistente.

Sem sincronização, dois depósitos separados de invocações podem ler o saldo antes de incrementá-lo, resultando num valor incorreto. Qualquer método que acesse uma variável de instância que possa variar deve ser sincronizado.

Operações isentas de interferência de operações simultâneas realizadas em outros segmentos são chamadas de operações atômicas. O uso de sincronização de métodos em **Java** é uma maneira de obter operações atômicas, mas, em outra programação de ambientes para servidores *multi-threaded*, as operações em objetos ainda precisam ter operações atômicas para manter seus objetos consistentes.

Isso pode ser conseguido pelo uso de qualquer mecanismo de exclusão mútua disponível, como um *mutex*.

## Cooperação do cliente por meio da sincronização das operações do servidor

---

Os clientes podem usar um servidor como meio de compartilhar alguns recursos. Isso é conseguido por alguns clientes usando operações para atualizar os objetos do servidor e por outros utilizando operações para acessá-los.

O acesso sincronizado aos objetos oferece o que é necessário para as aplicações funcionarem adequadamente e evita que *threads* (processos) interfiram uns com os outros.

No entanto, alguns aplicativos exigem uma maneira de os *threads* se comunicarem entre si. Por exemplo, pode surgir uma situação em que a operação solicitada por um cliente não possa ser concluída até que uma operação solicitada por outro cliente tenha sido executada.

Isso pode acontecer quando alguns clientes são produtores e outros são consumidores – os consumidores podem ter de esperar até que um produtor forneça mais do produto em questão; também pode ocorrer quando os clientes compartilham um recurso – os clientes que precisam de recurso podem ter de esperar outros clientes liberá-lo.

À frente, vamos ver que uma situação semelhante surge quando bloqueios ou registros de data e hora são usados para controle de simultaneidade em transações.

Considere a implementação de um objeto de fila compartilhado com dois métodos: inicialmente, remove e retorna o primeiro objeto na fila, e acrescenta um determinado objeto ao fim dela. No início, o método testará se a fila está vazia; nesse caso, ele vai chamar espera na fila.

Se um cliente carregar um processo antes de a fila ficar vazia, não será possível obter uma resposta até que outro cliente tenha retirado algo da fila de processos.

Isso permite que o encadeamento de objetos numa fila retorne o primeiro objeto para seu cliente. Quando os encadeamentos sincronizam suas ações num objeto por meio de espera, notificam o servidor e retêm os pedidos que não podem ser atendidos imediatamente, e o cliente aguarda uma resposta até que outro cliente tenha produzido o que for necessário.

Sem a capacidade de sincronizar *threads* dessa maneira, um cliente que não pode ser atendido imediatamente, por exemplo, um cliente que invoca o primeiro método numa fila, é orientado a tentar novamente mais tarde.

Isso é insatisfatório, porque envolverá o cliente na pesquisa do servidor na realização de solicitações extras; também é potencialmente injusto, porque outros clientes podem fazer seus pedidos antes de o cliente em espera tentar novamente.

## Modelo de falha para transações

Nesse modelo, a alegação é que os algoritmos funcionam corretamente na presença de falhas previsíveis, mas nenhuma reivindicação é feita sobre o seu comportamento quando ocorre um desastre. Embora erros possam ocorrer, eles podem ser detectados e tratados antes de qualquer comportamento incorreto.

As gravações no armazenamento permanente podem falhar, seja por escrever nada, seja por escrever o valor incorreto; por exemplo, escrever no bloco errado é um desastre.

O armazenamento de arquivo também pode cair. As leituras do armazenamento permanente podem detectar (por uma soma de verificação) quando um Bloco de Dados é ruim.

Quando um servidor com falha é substituído por um novo Processo, sua memória volátil é primeiro definida para um estado que não conhece nenhum dos valores (por exemplo, de objetos) de antes da falha.

Depois disso, realiza um procedimento de recuperação, utilizando informações armazenadas permanentemente e outros Processos para definir os valores dos objetos, incluindo aqueles relacionados às duas fases do Protocolo de Confirmação.

Um processador é feito para travar, de modo que, quando estiver com defeito, será impedido de enviar mensagens erradas e de escrever errado valores para armazenamento permanente, isto é, não pode produzir falhas arbitrárias.

Falhas podem ocorrer a qualquer momento; em particular, elas podem ocorrer durante a recuperação. Pode haver um atraso arbitrário antes que uma mensagem chegue; uma mensagem pode ser perdida, duplicada ou corrompida.

O destinatário pode detectar mensagens corrompidas usando um *checksum*. Tanto as mensagens forjadas, quanto as mensagens corrompidas não detectadas são consideradas desastres.

O modelo de falha para armazenamento permanente, processadores e comunicações foi usado para projetar um Sistema estável, cujos componentes possam sobreviver a qualquer falha e apresentar modelo de falha simples.

Em particular, o armazenamento estável forneceu uma operação de gravação atômica na presença de uma única falha da operação de gravação ou uma falha de falha do Processo, conseguida por meio da replicação de cada bloco em dois blocos de disco.

Uma operação de gravação foi aplicada ao par de blocos de disco e, no caso de uma única falha, um bom bloco estava sempre disponível. Um processador estável usou armazenamento estável para permitir a recuperação de objetos após uma falha; erros de comunicação foram mascarados, usando um controle remoto confiável num mecanismo de chamada de procedimento.

## Transações

Em algumas situações, os clientes requerem uma sequência de pedidos separados para um servidor, no sentido de que:

1. Eles estão livres de interferências por operações realizadas em nome de outros clientes simultâneos;
2. Todas as operações devem ser concluídas com sucesso ou não devem ter nenhum efeito na presença de falhas no servidor.

As transações são originadas de Sistemas de Gerenciamento de Banco de Dados. Nesse contexto, transação é uma execução de um Programa que acessa um Banco de Dados.

As transações foram introduzidas em Sistemas Distribuídos na forma de servidores de arquivos transacionais. No contexto de um servidor de arquivos transacional, uma transação é uma execução de uma sequência de pedidos de clientes para operações de arquivos.

Uma transação consiste na execução de uma sequência de solicitações de um determinado cliente.

Do ponto de vista do cliente, uma transação é uma sequência de operações que forma uma única etapa, transformando os dados do servidor de um estado consistente para outro.

Aos clientes são fornecidas operações para especificar o início e o fim de uma transação, que se propaga com cada operação nessa transação.

Em todos esses contextos, uma transação se aplica a objetos recuperáveis e se destina a ser atômica. Muitas vezes, é chamada de transação atômica.

Existem dois aspectos para a atomicidade:

- **Tudo ou nada:** uma transação é concluída com êxito, caso em que os efeitos de todas as suas operações são registrados nos objetos ou (se falhar ou for deliberadamente abortada) não tem efeito algum. Esse efeito – tudo ou nada – tem dois aspectos adicionais:
- **Atitude de falha:** os efeitos são atômicos mesmo quando o servidor falha;
- **Durabilidade:** depois que uma transação foi concluída com sucesso, todos os seus efeitos são salvos em armazenamento permanente. Usamos o termo “armazenamento permanente” para nos referirmos aos arquivos mantidos em disco ou outro meio permanente. Os dados salvos num arquivo sobreviverão se o Processo do servidor travar;
- **Isolamento:** cada transação deve ser realizada sem interferência de outras transações; em outras palavras, os efeitos intermediários de uma transação não devem ser visíveis para outras transações.

Para suportar o requisito de falha de atomicidade e durabilidade, os objetos devem ser recuperáveis, isto é, quando um Processo do servidor falhar inesperadamente, devido a uma falha de *hardware* ou a um erro de *software*, as alterações devidas a todas as transações concluídas devem estar disponíveis no armazenamento permanente para que, quando o servidor for substituído por um novo Processo, os objetos possam refletir o efeito tudo ou nada.

No momento em que um servidor reconhece a conclusão da transação de um cliente, todas as alterações da transação dos objetos devem ser gravadas em armazenamento permanente.

Um servidor que suporta transações deve sincronizar as operações o suficiente para garantir que o requisito de isolamento seja atendido. Uma maneira de fazer isso é executar transações em série – uma de cada vez, em alguma ordem arbitrária.

Infelizmente, essa solução, geralmente, seria inaceitável para servidores cujos recursos são compartilhados por vários usuários interativos. Em um Sistema Bancário, por exemplo, é desejável permitir que vários funcionários que trabalhem no Banco realizem transações bancárias *on-line* ao mesmo tempo em que outros funcionários.

O objetivo de qualquer servidor que suporte transações é maximizar a simultaneidade; portanto, as transações podem ser executadas simultaneamente se isso tiver o mesmo efeito que uma execução serial, isto é, se elas são serialmente equivalentes ou serializáveis.

Uma transação é obtida pela cooperação entre um programa cliente, alguns objetos recuperáveis e um coordenador. O cliente especifica a sequência de invocações em objetos recuperáveis que devem compor uma transação.

Normalmente, uma transação é concluída quando o cliente faz um pedido de finalização de transação. Se a transação progrediu normalmente, a resposta afirma que a transação está comprometida – isso constitui uma promessa ao cliente, de que todas as mudanças solicitadas na transação serão registradas permanentemente e que quaisquer transações futuras que acessam os mesmos dados verão os resultados de todas as alterações feitas durante a transação.

Alternativamente, a transação pode ter de abortar por uma das várias razões relacionadas à própria natureza da transação, a conflitos com outra transação ou à falha de um Processo ou computador.

Quando uma transação é abortada, as partes envolvidas devem garantir que nenhum de seus efeitos seja visível para transações futuras, seja nos objetos, seja em suas cópias em armazenamento permanente.

Uma transação é bem-sucedida ou é cancelada de duas maneiras: o **cliente cancela a operação** ou o **servidor cancela a operação**.



Se um processo do servidor falhar inesperadamente, será eventualmente substituído. O novo processo do servidor anula todas as transações não confirmadas e usa um procedimento de recuperação para restaurar os valores dos objetos para os valores produzidos pela transação mais recentemente confirmada.

Para lidar com um cliente que falha inesperadamente durante uma transação, os servidores podem dar a cada transação um tempo de expiração e abortar uma transação que não foi concluída antes de seu tempo de expiração.

Se um servidor travar enquanto uma transação estiver em andamento, o cliente ficará ciente disso quando uma das operações retornar uma exceção após um tempo limite.

Se um servidor falhar e for substituído durante o progresso de transação, ela não será mais válida e o cliente deve ser informado via uma exceção à próxima operação.

Em ambos os casos, o cliente deve formular um plano, possivelmente em consulta com o usuário humano, para a conclusão ou o abandono da tarefa da qual a transação fazia parte.

### Controle de concorrência

Este item descreve dois problemas bem conhecidos de transações simultâneas no contexto de um exemplo bancário: o problema de **atualização perdida** e o **inconsistente problema de recuperação**.

Se todas as transações forem conhecidas por terem o efeito correto quando são feitas por conta própria, então, podemos inferir que, se essas transações são feitas, uma a uma, em alguma ordem, o efeito combinado também estará correto.

Uma intercalação de operações de transações em que o efeito combinado é o mesmo que se as transações fossem realizadas uma de cada vez, em alguma ordem, é uma intercalação em série equivalente.

Quando dizemos que duas transações diferentes têm o mesmo efeito uma que a outra, significa que as operações de leitura retornam aos mesmos valores e que as variáveis de instância dos objetos têm os mesmos valores no final.

O uso da equivalência serial como critério para a correta execução concorrente impede a ocorrência de atualizações perdidas e recuperações inconsistentes.

O problema de atualização perdida ocorre quando duas transações leem o valor antigo de uma variável e depois o usam para calcular o novo valor. Isso não pode acontecer se uma transação for realizada antes de outra, porque a transação posterior irá ler o valor escrito pela anterior.

Quando dizemos que um par de operações conflitam significa que seu efeito combinado depende da ordem em que são executadas.

Para qualquer par de transações, é possível determinar a ordem dos pares de operações conflitantes em objetos acessados por ambos. A equivalência serial pode ser definida em termos de conflitos de operação, da forma descrita a seguir.

Para que duas transações sejam equivalentes em série, é necessário e suficiente que pares de operações conflitantes das duas transações sejam executadas na mesma ordem, em todos os objetos que ambas acessam.

Basicamente, o controle de concorrência pode ser alcançado pelas transações dos clientes esperando um pelo outro ou reiniciando transações após conflitos que foram detectados entre operações, ou por uma combinação dos dois.

### Transações perdidas

Os servidores devem registrar todos os efeitos das transações confirmadas e nenhum dos efeitos das transações abortadas; devem, portanto, permitir que uma transação possa abortar, impedindo que ela afete outras transações simultâneas se o fizer.

Este item ilustra dois problemas associados ao abortamento de transações no contexto do exemplo bancário. Esses problemas são chamados de **leituras sujas** e **prematuras**.

Ambos podem ocorrer na presença de execuções equivalentes em série de transações. Essas questões estão relacionadas aos efeitos das operações em objetos, tais como o saldo de uma conta bancária. Para simplificar as coisas, as operações são consideradas em duas categorias: **operações de leitura** e **operações de gravação**.

A propriedade de isolamento das transações exige que as transações não vejam o estado de não comprometido de outras transações. O problema da **leitura suja** é causado pela interação entre uma operação de leitura em uma transação e uma operação de gravação em outra, no mesmo objeto.

Se uma transação qualquer foi confirmada depois de ter visto os efeitos de uma transação que falhou, a situação não é recuperável e pode ser definida como uma transação prematura.

A estratégia é garantir que a transação apenas ocorra no caso de sua finalização, isto é, quando o Processo confirma a gravação do dado por completo.

Considere outra implicação na possibilidade de uma transação falhar, que está relacionada à interação entre as operações de gravação no mesmo objeto pertencente a diferentes transações.

Alguns Sistemas de Banco de Dados implementam a ação de abortar uma ação quando ela não foi finalizada, realizando seu estorno por completo. Geralmente, é necessário que as transações atrasem suas operações de leitura e gravação, de modo a evitar problemas e gravações prematuras.

Para que um servidor de objetos recuperáveis participe de transações, deve ser projetado para que quaisquer atualizações de objetos possam ser removidas se e quando a transação for anulada.

Para tornar isso possível, todas as operações de atualização realizadas durante uma transação devem ser feitas em versões experimentais de objetos, na memória volátil.

Cada transação é fornecida com seu próprio conjunto particular de versões provisórias de quaisquer objetos que alterou. Todas as operações de atualização de uma transação armazenam valores na transação do próprio conjunto privado.

As operações de acesso numa transação recebem valores da transação próprios do conjunto privado, se possível ou, na sua falta, a partir dos objetos.

As versões provisórias são transferidas para os objetos apenas quando uma transação é confirmada, no momento em que elas são gravadas no armazenamento permanente. Isso é realizado numa única etapa, durante a qual outras transações são excluídas do acesso aos objetos que estão sendo alterados; quando uma transação é anulada, suas versões experimentais são excluídas.

### Transações aninhadas

As transações aninhadas estendem o modelo de transação, permitindo que as transações sejam compostas de outras transações. Assim, várias transações podem ser iniciadas dentro de uma transação, permitindo que transações sejam consideradas módulos que podem ser requeridos.

A transação mais externa num conjunto de transações aninhadas é chamada de **nível superior de transação**; transações além da transação de nível superior são chamadas de **subtransações**.

Se uma ou mais subtransações falharem, a transação pai poderá registrar o fato e, em seguida, verificar o resultado de todas as transações filhas e se foram bem-sucedidas. Isso poderia, então, iniciar outra transação para tentar entregar novamente as mensagens que não foram enviadas na primeira vez.

Quando precisamos distinguir nossa forma original de transação das transações aninhadas, usamos o termo **transação plana**. É plana porque todo o seu trabalho é feito no mesmo nível entre um *commit* ou *abort*.

Transações aninhadas têm as seguintes vantagens principais:

1. As subtransações num nível (e seus descendentes) podem ser executadas simultaneamente a outras subtransações no mesmo nível na hierarquia;
2. As subtransações podem comprometer ou anular de forma independente; em comparação a uma única transação, um conjunto de subtransações aninhadas é potencialmente mais robusto.

As regras para o envio de transações aninhadas são bastante sutis:

- Uma transação pode ser confirmada ou anulada somente depois que suas transações estiverem concluídas;
- Quando uma subtransação está completa, ela toma uma decisão independente para gravar provisoriamente ou cancelar a subtransação;
- Quando uma transação pai é anulada, todas as suas subtransações são anuladas;
- Quando uma subtransação aborta, a transação pai pode decidir se aborta ou não.

Em alguns casos, a transação de nível superior pode decidir abortar porque uma ou mais de suas subtransações foram anuladas.

O **CORBA** – *Object Transaction Service* suporta planos de transações aninhadas, que são particularmente úteis em Sistemas Distribuídos porque transações-filhas podem ser executadas simultaneamente em diferentes servidores.

## Bloqueios

As transações devem ser agendadas para que seu efeito em série nos dados compartilhados seja equivalente. Um servidor pode alcançar a equivalência serial de transações, serializando o acesso aos objetos.

Um exemplo simples de um mecanismo de serialização é o uso de bloqueios exclusivos.

Nesse esquema de bloqueio, o servidor tenta bloquear qualquer objeto que esteja prestes a ser usado na operação da transação de um cliente. Se um cliente solicitar acesso a um objeto que já esteja bloqueado devido à transação de outro cliente, a solicitação é suspensa e o cliente deve aguardar até que o objeto esteja desbloqueado.

Transações podem abortar execuções estritas quando necessário para evitar leituras sujas e gravações prematuras. Sob um regime estrito de execução, uma transação que precisa ler ou escrever um objeto deve ser adiada até que outras transações que gravaram o mesmo objeto possam ser confirmadas ou anuladas.

Para impor essa regra, quaisquer bloqueios aplicados durante o andamento de uma transação são mantidos até que a transação seja finalizada ou tenha ocorrido uma falha.

Isso é chamado de bloqueio estrito de duas fases. A presença dos bloqueios impede que outras transações leiam ou escrevam os objetos.

Para garantir a capacidade de recuperação, os bloqueios devem ser mantidos até que todos os objetos atualizados sejam escritos para armazenamento permanente.

Um servidor, geralmente, contém um grande número de objetos e uma transação típica acessa apenas alguns deles, e é improvável que colida com outras transações atuais.

Os esquemas de Controle de Concorrência não pressupõem nenhuma granularidade em particular. Os Protocolos de Controle de Concorrência são aplicáveis aos objetos cujas operações podem ser definidas em operações de leitura e gravação.

Os Protocolos de Controle de Concorrência são projetados para lidar com conflitos entre operações em diferentes transações no mesmo objeto; portanto, um simples bloqueio exclusivo, que é usado para operações de leitura e gravação, reduz a concorrência mais do que é necessário.

É preferível adotar um esquema de bloqueio que controle o acesso a cada objeto em que pode haver várias transações simultâneas, lendo um objeto ou uma única transação escrevendo um objeto, mas não ambos.

Isso é comumente chamado de “muitos esquemas de leitores/escritores únicos”. Dois tipos de bloqueios são usados: **bloqueios de leitura** e **bloqueios de gravação**.

Antes que a operação de leitura de uma transação seja realizada, um bloqueio de leitura deve ser definido no objeto; antes que a operação de gravação de uma transação seja executada, um bloqueio de gravação deve ser definido no objeto.

Sempre que for impossível definir um bloqueio imediatamente, a transação (e o cliente) deve esperar até que seja possível fazê-lo – a solicitação de um cliente nunca é rejeitada.

Como pares de operações de leitura de transações diferentes não entram em conflito, uma tentativa de definir um bloqueio de leitura num objeto com um bloqueio de leitura é sempre bem-sucedida. Todas as transações podem ler o mesmo objeto e compartilhar seu bloqueio de leitura, esses bloqueios de leitura são, por vezes, chamados de **bloqueios compartilhados**.

As regras de conflito de operação nos dizem que:

1. Se uma transação T já realizou uma operação de leitura num objeto específico, então, uma transação concorrente U não deve escrever esse objeto até T concluir ou abortar;
2. Se uma transação T já realizou uma operação de gravação num objeto específico, então, uma transação simultânea U não deve ler ou escrever aquele objeto até T concluir ou anular.

Para impor a condição 1, uma solicitação para um bloqueio de gravação num objeto é atrasada pela presença de um bloqueio de leitura pertencente a outra transação.

Para reforçar a condição 2, um pedido de um bloqueio de leitura ou um bloqueio de gravação num objeto é atrasado pela presença de um bloqueio de gravação pertencente a outra transação.

Uma transação com um bloqueio de leitura compartilhado com outras transações não pode promover seu bloqueio de leitura para um bloqueio de gravação, porque o último entraria em conflito com as travas de leitura detidas pelas outras transações; portanto, essa transação deve solicitar um bloqueio de gravação e aguardar até que os outros bloqueios de leitura sejam liberados.

Bloquear promoção refere-se à conversão de um bloqueio num bloqueio mais forte, isto é, bloqueio que é mais exclusivo. O bloqueio de leitura permite outros bloqueios de leitura, enquanto a gravação do bloqueio não – sequer permite outros bloqueios de gravação; portanto, um bloqueio de gravação é mais exclusivo do que um bloqueio de leitura.

Bloqueios podem ser promovidos porque o resultado é um bloqueio mais exclusivo.

Não é seguro manter uma transação desbloqueada antes de ela ser finalizada, pois o resultado final pode ser inadequado e poderão acontecer outras transações inconsistentes.

## Regras para o uso de bloqueios

O bloqueio é executado quando os pedidos de operações de leitura e gravação estão prestes a serem aplicados a objetos recuperáveis e o desbloqueio é realizado pelas operações de submissão ou anulação da transação Coordenador.

Por exemplo, o Serviço de Controle de Concorrência do CORBA pode ser usado para aplicar o controle de concorrência em transações ou proteger objetos sem usar transações.

A concessão de bloqueios será implementada por um objeto separado no servidor, que chamamos de gerenciador de bloqueio, que mantém um conjunto de travas, por exemplo, numa Tabela de *hash*.

Cada bloqueio é uma instância da classe *Lock* e está associado a um objeto particular e cada instância do bloqueio mantém as seguintes informações em suas variáveis de instância:

- O identificador do objeto bloqueado;
- Os identificadores das transações que atualmente mantêm o bloqueio;
- O tipo de bloqueio.

Os métodos de bloqueio são sincronizados para que os encadeamentos que tentam adquirir ou liberar um bloqueio não interfiram com o outro; mas, além disso, tenta adquirir o bloqueio usando o método de espera sempre que ele precisa aguardar que outro segmento o libere.



## Controle de concorrência

A manutenção de bloqueio representa uma sobrecarga que não está presente em Sistemas que não suportem acesso simultâneo a dados compartilhados.

Mesmo as transações de leitura, que não podem afetar a integridade dos dados, devem, em geral, usar o bloqueio, a fim de garantir que os dados que estão sendo lidos não sejam modificados por outras transações ao mesmo tempo; mas o bloqueio pode ser necessário apenas no pior dos casos.

O controle de concorrência contribui em Sistemas Distribuídos evitando, assim, problemas de falhas em transações.

### Comparação de métodos para controle de concorrência

Existem alguns métodos para controlar o acesso simultâneo a dados: **bloqueio estrito de duas fases, métodos otimistas e ordenação de registro de data e hora.**

Todos os métodos carregam algumas despesas gerais no tempo e no espaço que eles exigem, e todos eles, em certa medida, o potencial para operação simultânea.

Os mecanismos de Controle de Concorrência acima não são sempre adequados para aplicativos do século XXI, que permitem aos usuários compartilhar documentos na *Internet*. Muitos usam formas otimistas de controle de simultaneidade seguidas por resolução de conflitos em vez de abortar um de qualquer par de operações conflitantes.

A seguir, alguns exemplos:

- **Dropbox ([www.dropbox.com](http://www.dropbox.com)):** é um serviço em nuvem que fornece *backup* de arquivos e permite que os usuários compartilhem arquivos e pastas, acessando-os de qualquer lugar. Assim, se dois usuários fizerem atualizações simultâneas para o mesmo arquivo, a primeira gravação será aceita e a segunda rejeitada. No entanto, o Dropbox fornece um histórico de versão para permitir que os usuários mesquem suas atualizações manualmente ou restaurem versões anteriores;
- **Google Apps (incluem o Google Docs):** é um serviço de nuvem que fornece aplicativos baseados no Web processador, planilha e apresentação, que permitem ao usuário colaborar com outro por meio de documentos compartilhados. Se várias pessoas editarem o mesmo documento simultaneamente, verão as alterações umas das outras. No caso de um documento no processador de texto, os usuários podem ver os cursores e as atualizações dos outros são mostradas no nível de caracteres individuais, haja vista que são digitados por qualquer participante. Os usuários são deixados para resolver quaisquer conflitos que ocorrem, mas, geralmente, os conflitos são evitados porque os usuários estão continuamente conscientes das atividades uns dos outros. No caso de um documento de planilha, os cursores dos usuários e as alterações são exibidas e atualizadas na granularidade de células individuais. Se dois usuários acessarem a mesma célula simultaneamente, a última atualização ganha;

- **Wikipedia:** o controle de concorrência para edição é otimista, permitindo que os editores concomitantes acessem as páginas da *web* em que a primeira gravação é aceita, e um usuário fazendo uma *write* é exibido numa tela de “conflito de edição” e chamado para a resolução dos conflitos.

## Resumo

---

As transações fornecem um meio pelo qual os clientes podem especificar sequências de operações que são atômicas na presença de outras transações simultâneas e falhas de servidor.

O primeiro aspecto da atomicidade é conseguido executando transações, de modo que seus efeitos sejam serialmente equivalentes. Os efeitos das transações confirmadas são registrados em armazenamento permanente em que o serviço de transação pode se recuperar de falhas de Processo.

Para permitir transações, a capacidade de abortar, sem ter efeitos colaterais prejudiciais em outras transações e execuções, deve ser estrita, isto é, as leituras e as gravações de uma transação devem ser postergadas até que outras transações que gravaram os mesmos objetos sejam confirmadas ou anuladas.

A tentativa de versões de objetos é copiada para os objetos reais e para o armazenamento permanente na confirmação de transação.

As transações aninhadas são formadas pela estruturação de transações de outros transações, sendo que o aninhamento é particularmente útil em Sistemas Distribuídos porque permite execução simultânea de subtransações em servidores separados. O aninhamento também tem a vantagem de permitir a recuperação independente de partes de uma transação.

Conflitos de operação formam uma base para a derivação do controle de concorrência de Protocolos, que não devem apenas garantir a serialização, mas também permitir a recuperação, usando execuções estritas para evitar problemas associados ao abortamento de transações, como anulações em cascata.

Três alternativas de estratégias são possíveis no agendamento de uma operação numa transação: (1) **executá-la imediatamente**; (2) **atrasá-la** ou (3) **abortá-la**.

O bloqueio estrito de duas fases usa as duas primeiras estratégias, recorrendo ao aborto apenas no caso de impasse e assegura a serialização ao ordenar transações de acordo com quando elas foram acessadas; sua principal desvantagem é que podem ocorrer *deadlocks*.

A ordenação de registro de data e hora usa todas as três estratégias para garantir a serialização por meio de pedidos e acessos das transações aos objetos, de acordo com o tempo de início das transações.

Esse método não pode sofrer de *deadlocks* e é vantajoso somente para transações de leitura; contudo, as transações devem ser canceladas quando chegarem tarde demais.

Ordenação de *timestamp* multiversão são particularmente eficazes.

O controle de concorrência otimista permite que as transações prossigam sem qualquer forma de verificação até que estejam concluídas.

A validação reversa requer a manutenção de vários conjuntos de gravação de transações confirmadas, enquanto a validação direta deve ser validada em relação a transações e tem a vantagem de permitir estratégias alternativas para a resolução de conflitos.

O problema pode ocorrer devido a repetidos abortos de uma transação que falha na validação do controle de simultaneidade otimista e até mesmo na ordenação de *timestamp*.

## Transações Distribuídas

Este item apresenta as transações distribuídas – aquelas que envolvem mais de um servidor.

Transações distribuídas podem ser planas ou aninhadas.

Um Protocolo de Confirmação Atômica é um procedimento cooperativo usado por um conjunto de servidores envolvidos numa transação distribuída e permite que os servidores cheguem a uma decisão conjunta se uma transação pode ser confirmada ou anulada.

Este item descreve as duas fases do Protocolo de Confirmação, que é o Protocolo de Confirmação Atômica mais comumente usado.

A seção sobre controle de concorrência em transações distribuídas discute como bloqueio o pedido de *timestamp* e o controle de simultaneidade otimista, que podem ser estendidos para uso com transações distribuídas.

Os servidores que fornecem transações incluem um gerenciador de recuperação cuja preocupação é garantir que os efeitos das transações nos objetos gerenciados por um servidor possam ser recuperados quando forem substituído após uma falha.

O gerenciador de recuperação salva os objetos no armazenamento permanente juntamente com listas de intenções e informações sobre o *status* de cada transação.

### Transações que acessavam objetos num único servidor – Introdução

---

No caso de uma transação, seja plana ou aninhada, ela acessará objetos localizados em vários computadores diferentes.

Usamos o termo transação distribuída para nos referirmos a uma transação simples ou aninhada que acessa objetos gerenciados por vários servidores.

Quando uma transação distribuída chega ao fim, a propriedade das transações requer que todos os servidores envolvidos finalizem a transação ou que todos eles a abortem.

Para conseguir isso, um dos servidores assume a função de coordenador, o que envolve garantir o mesmo resultado em todos os servidores. A maneira como o Coordenador consegue isso depende do Protocolo escolhido.

Um Protocolo conhecido como **Protocolo de Confirmação de Duas Fases** é o mais comumente usado. Esse Protocolo permite aos servidores que se comuniquem entre si para chegar a uma decisão conjunta sobre finalizar ou abortar.

A recuperação de transações se preocupa em garantir que todos os objetos envolvidos nas transações são recuperáveis.

Além disso, garante que os valores dos objetos reflitam todas as alterações feitas por transações confirmadas e nenhuma das feitas pelas abortadas.

## Transações distribuídas planas e aninhadas

Uma transação do cliente é distribuída se invocar operações em vários servidores. Existem duas maneiras diferentes pelas quais as transações distribuídas podem ser estruturadas: como **transações simples** e como **transações aninhadas**.

Quando os servidores usam o bloqueio, uma transação pode estar aguardando apenas um objeto por vez. Numa transação aninhada, a transação de nível superior pode abrir subtransações e cada subtransação pode abrir subtransações adicionais, até qualquer profundidade de aninhamento.

### O Coordenador de uma transação distribuída

Servidores que executam solicitações como parte de uma transação distribuída precisam ser capazes de se comunicar entre si para coordenar suas ações quando a transação for confirmada.

Identificadores de transação para transações distribuídas devem ser exclusivos dentro de um Sistema. Uma maneira simples de conseguir isso é um **TID** conter duas partes: o identificador exemplo, um endereço **IP** do servidor que o criou e um número exclusivo para o servidor.

O Coordenador que abriu a transação passa a ser o Coordenador da transação distribuída e, no final, é responsável por comprometê-la ou abortá-la.

Cada um dos servidores que gerencia um objeto acessado por uma transação é um participante da transação e fornece um objeto que chamamos de participante.

Cada participante é responsável por manter o controle na transação de todos os objetos recuperáveis nesse servidor em que estão envolvidos. Os participantes são responsáveis por cooperar com o coordenador na execução do Protocolo de Confirmação.

## Protocolos de Confirmação

Os Protocolos de Transação foram criados no início dos anos 1970, e as duas fases do Protocolo de Confirmação surgiram a partir de 1978.

A propriedade de atomicidade das transações exige que, quando uma transação distribuída chega ao fim, todas as suas operações ou nenhuma delas sejam realizadas.

No caso de uma transação distribuída, o cliente solicita uma operação em mais de um servidor. Uma transação chega ao fim quando o cliente solicita que seja confirmada a gravação ou que ela seja abortada.

Uma maneira simples de concluir a transação de maneira atômica é o coordenador de processos comunicar o pedido de confirmação ou abortar todos os participantes da transação e continuar repetindo a solicitação até que todos eles reconheçam que ela já foi realizada; esse é um exemplo de um Protocolo de Confirmação Atômica de fase.

O Protocolo de Confirmação Atômica Monofásica é inadequado, porque não permite que um servidor tome a decisão unilateral de abortar uma transação quando o cliente solicita um *commit*.

Razões que impedem que um servidor consiga comprometer parte de uma transação geralmente estão relacionadas a questões de controle de concorrência. Por exemplo, se o bloqueio está em uso, a resolução de um impasse pode levar ao aborto de uma transação sem o cliente estar ciente, a menos que faça outra solicitação ao servidor.

Ademais, se o controle de concorrência otimista estiver em uso, a falha de validação num servidor causaria abortar a transação.

Finalmente, o Coordenador pode não saber se um servidor travou e foi substituído durante o progresso de uma transação distribuída – esse servidor terá de abortar a transação.

O Protocolo de Confirmação de Duas Fases é projetado para permitir que qualquer participante aborte parte de uma transação. Devido à exigência de atomicidade, se uma parte de uma transação for abortada, toda a transação deve ser abortada.

Na primeira fase do Protocolo, cada participante vota para a transação ser confirmada ou abortada; uma vez que um participante votou para finalizar uma transação, não é permitido abortá-la; portanto, antes de um participante escolher realizar uma transação, deve garantir que será capaz de executar sua parte do Protocolo de Confirmação, mesmo se falhar e for substituído nesse ínterim.

Um participante que esteja numa transação precisa estar preparado para ela se for, eventualmente, optar por finalizá-la. Para ter certeza disso, cada participante economiza armazenamento permanente de todos os objetos que ele alterou na transação, juntamente com seu *status*: preparado.

Na segunda fase do Protocolo, cada participante da transação realiza a decisão conjunta. Se algum participante votar para abortar, então, a decisão deve ser a de abortar a transação. Se todos os participantes votarem para que a transação seja finalizada, então a decisão é finalizar a transação.

O problema é garantir que todos os participantes votem e que todos tomem a mesma decisão. Isso é bastante simples se não ocorrerem erros, mas o Protocolo deve funcionar corretamente, mesmo quando alguns servidores falham, as mensagens são perdidas ou os servidores ficam temporariamente incapazes de se comunicar uns com os outros.

Os Protocolos de Confirmação são projetados para funcionar num Sistema Assíncrono em que os servidores podem falhar e as mensagens podem ser perdidas. Supõe-se que uma solicitação-resposta subjacente ao Protocolo remove mensagens corrompidas e duplicadas. Não há falhas bizantinas – servidores falham ou obedecem às mensagens que são enviadas.

### Controle de concorrência em transações distribuídas

Cada servidor gerencia um conjunto de objetos e é responsável por garantir que eles permaneçam consistentes quando acessados por transações simultâneas; portanto, cada servidor é responsável por aplicar o controle de concorrência a seus próprios objetos. Os membros de uma recolha de servidores de transações distribuídas são solidariamente responsáveis por garantir que elas sejam realizadas de maneira serialmente equivalentes.

### Bloqueio

Numa transação distribuída, os bloqueios num objeto são mantidos localmente; o gerenciador de bloqueio local pode decidir se concederá uma trava ou fará o pedido de espera de transação. No entanto, ele não pode liberar nenhum bloqueio até saber que a transação foi confirmada ou abortada em todos os servidores envolvidos na transação.

### Controle de simultaneidade de ordenação de registro de data e hora

Numa transação de servidor único, o Coordenador emite um registro de data e hora exclusivo para cada transação, quando ela é iniciada. A equivalência serial é reforçada ao se comprometer com as versões objeto na ordem dos *timestamps* das transações que os acessaram.

Em transações distribuídas, exigimos que cada coordenador emita *timestamps* globalmente exclusivos. Um *timestamp* de transação globalmente exclusivo é emitido para o cliente pelo primeiro Coordenador acessado por uma transação.

O registro de data e hora da transação é passado para o Coordenador em cada servidor cujos objetos executam uma operação na transação.



## Controle de concorrência otimista

Lembre-se de que, com o controle de concorrência otimista, cada transação é validada antes que haja permissão para se comprometer. Os números de transação são atribuídos no início da validação e as transações são serializadas de acordo com a ordem dos números de transação.

Uma transação distribuída é validada por uma coleção de servidores independentes, cada um dos quais valida transações que acessam seus próprios objetos. Essa validação ocorre durante a primeira fase do Protocolo de Confirmação de Duas Fases.

## Deadlocks distribuídos

Servidores devem evitar ou detectar e resolver impasses. Usar *timeouts* para resolver possíveis *deadlocks* é uma abordagem desajeitada – é difícil escolher um intervalo de tempo limite adequado e transações podem ser anuladas desnecessariamente.

Com esquemas de detecção de *deadlock*, a transação é anulada somente quando está envolvida num *deadlock*.

## Recuperação de transação

A propriedade atômica das transações requer que todos os efeitos de transações e nenhum dos efeitos de transações incompletas ou abortadas sejam refletidos nos objetos que eles acessaram.

Essa propriedade pode ser descrita em dois aspectos: **durabilidade** e **falha de atomicidade**.

A **durabilidade** requer que os objetos sejam salvos em armazenamento permanente e estará disponível indefinidamente a partir de então; portanto, um reconhecimento da solicitação de confirmação de um cliente implica que todos os efeitos da transação foram gravados em armazenamento permanente, bem como nos objetos (voláteis) do servidor. A atomicidade exige que os efeitos das transações sejam atômicos, mesmo quando o servidor falha.

A recuperação se preocupa em garantir que os objetos de um servidor sejam duráveis e que os serviços fornecem atomicidade de falha.

Embora os servidores de arquivos e os servidores de Banco de Dados mantenham os dados em armazenamento permanente, outros tipos de servidores de objetos recuperáveis não precisam fazê-lo, exceto para a recuperação de finalidades.

Assumimos que, quando um servidor está em execução, ele mantém todos os seus objetos em sua memória volátil e registra seus objetos comprometidos num arquivo ou arquivos de recuperação; portanto, a recuperação consiste em restaurar o servidor com o último *commit* das versões de seus objetos de armazenamento permanente.

Bancos de Dados precisam lidar com grandes volumes de Dados; eles geralmente mantêm os objetos em armazenamento estável no disco, com um *cache* na memória volátil.

## Logging

Na técnica de *log*, o arquivo de recuperação representa um *log* contendo o histórico de todas as transações realizadas por um servidor. A história consiste em valores de objetos, entradas de *status* de transação e listas de intenções de transação.

A ordem das entradas no *log* reflete a sequência da ordem na qual as transações foram preparadas, confirmadas e abortadas naquele servidor. Na prática, o arquivo de recuperação conterá um instantâneo recente dos valores de todos os objetos no servidor, seguido por um histórico de transações posteriores à captura instantânea.

Durante a operação normal de um servidor, seu gerenciador de recuperação é sempre chamado numa transação e se prepara para confirmá-la ou para interrompê-la.

Quando o servidor está preparado para enviar uma transação, o gerenciador de recuperação anexa todos os objetos em suas intenções para o arquivo de recuperação, seguido pelo *status* atual da transação (preparado), juntamente com a sua lista de intenções.

Quando uma transação é eventualmente confirmada ou anulada, o gerenciador de recuperação acrescenta o *status* correspondente da transação a seu arquivo de recuperação.

Assume-se que a operação de acréscimo é atômica no sentido em que escreve uma ou mais entradas completas para o arquivo de recuperação. Se o servidor falhar, somente a última gravação poderá ser incompleta.

Para fazer uso eficiente do disco, várias gravações subsequentes podem ser armazenadas em *buffer* e, em seguida, gravadas no disco como uma única gravação. Uma vantagem adicional do registro da técnica é que as gravações sequenciais em disco são mais rápidas do que as feitas em locais aleatórios.

Após uma falha, qualquer transação que não tenha *status* confirmado no *log* é abortada; portanto, quando uma transação é confirmada, sua entrada de *status* confirmada deve ser forçada para o *log* – isto é, gravada no *log* junto com qualquer outra entrada em *buffer*.

## Resumo

---

No caso mais geral, a transação de um cliente solicitará operações em objetos em vários servidores diferentes.

Uma transação distribuída é qualquer transação cuja atividade envolve vários servidores diferentes e uma estrutura de transação aninhada pode ser usada para permitir concorrência adicional e o comprometimento independente pelos servidores numa distribuição de transação.

A propriedade de atomicidade das transações requer que os servidores que participam de uma transação distribuída, todos, façam um *commit* ou todos a abortem.

Protocolos de *Commit Atomic* são projetados para obter esse efeito, mesmo se os servidores falharem durante sua execução.

Os Protocolos de Finalização permitem que um servidor decida abortar unilateralmente e incluem um tempo limite de ações para lidar com atrasos devido à queda de servidores.

O Protocolo de Confirmação de Duas Fases pode levar uma quantidade ilimitada de tempo para se completar, mas, eventualmente, será concluído.

O controle de concorrência em transações distribuídas é modular – cada servidor é responsável pela serialização de transações que acessam seus próprios objetos; contudo, Protocolos adicionais são necessários para garantir que as transações sejam serializáveis globalmente.

Transações distribuídas que usam ordenação de registro de data e hora requerem um meio para gerar ordem de data e hora entre os vários servidores.

Aqueles que usam Controle Otimista de Concorrência exigem validação global ou um meio de forçar um pedido global para finalizar as transações.

Transações distribuídas que usam bloqueio de duas fases podem sofrer com *deadlocks*. O objetivo da detecção de *deadlock* distribuído é procurar ciclos no Gráfico global de espera. Se um ciclo for encontrado, uma ou mais transações devem ser abortadas para resolver o impasse.

Aplicativos baseados em transações têm fortes requisitos para a longa vida útil e a integridade das informações armazenadas, mas, geralmente, não têm requisitos para resposta imediata em todos os momentos.

Protocolos de Confirmação Atômica são a chave para distribuir transações, mas não é garantido que elas sejam concluídas dentro de um limite de tempo específico.

As transações são duráveis, realizando *checkpoints* e registrando uma recuperação do arquivo, que é usado para recuperação quando um servidor é substituído após uma falha. Usuários de um serviço de transações sofrerão algum atraso durante a recuperação.

Embora se assuma que os servidores de transações distribuídas exibam falhas de pane e executem um Sistema Assíncrono, são capazes de chegar a um consenso sobre o resultado das transações porque os servidores com falha são substituídos por novos Processos que podem adquirir todas as informações relevantes de armazenamento permanente ou de outros servidores.

## Replicação

A **replicação** é a chave para fornecer alta disponibilidade e tolerância a falhas em Sistemas. A alta disponibilidade é de crescente interesse, com a tendência de mobilidade de computação e consequente operação desconectada.

A tolerância a falhas é uma permanente preocupação com os serviços prestados em Sistemas críticos de Segurança e em outros importantes.

Na primeira parte deste item, apresentam-se Sistemas que aplicam uma única operação num tempo para coleções de objetos replicados. Começa com uma descrição da arquitetura de componentes e um modelo de Sistema para serviços que empregam replicação.

Descrevemos a implementação da Gestão de membros de grupo como parte da Comunicação de grupo, o que é particularmente importante para serviços tolerantes a falhas.

## Introdução

---

Neste item, estudaremos a replicação de dados: a manutenção de cópias de dados em vários computadores. A replicação é a chave para a eficácia dos Sistemas Distribuídos nos quais se pode fornecer desempenho aprimorado, alta disponibilidade e tolerância a falhas.

A replicação é usada amplamente. Por exemplo, o armazenamento em cache de recursos de servidores da *Web*, navegadores e servidores *proxy* da *Web* é forma de replicação, já que os dados mantidos em *caches* e nos servidores são réplicas uns dos outros.

O serviço de nomes **DNS** mantém cópias de mapeamentos de nome para atributos para computadores e é confiável para o acesso diário a serviços pela *Internet*.

## Modelo do Sistema e papel da Comunicação em grupo

---

Os dados em nosso Sistema consistem numa coleção de itens que chamaremos de objetos.

**Objeto** pode ser um arquivo, digamos, ou um objeto Java, mas cada um desses objetos lógicos é implementado por uma coleção de cópias físicas chamadas réplicas.

As **réplicas** são objetos físicos, cada um armazenado num único computador, com dados e comportamentos ligados a alguns graus de consistência pela operação do Sistema.

As réplicas de um determinado objeto não são necessariamente idênticas, pelo menos não num determinado ponto no tempo; algumas réplicas podem ter recebido atualizações que outras não receberam.

Neste item, fornecemos um modelo geral de Sistema para gerenciar réplicas e, em seguida, descrevemos o papel dos Sistemas de Comunicação em grupo no alcance da tolerância a falhas de replicação, destacando a importância da comunicação de Grupos Síncronos de vista.

## Resumo

---

Replicar objetos é um meio importante de obter serviços com bom desempenho e alta disponibilidade e tolerância a falhas num Sistema Distribuído.

Descrevemos arquiteturas para serviços nos quais os gerentes de réplica possuem réplicas de objetos e em quais *front-ends* tornem essa replicação transparente.

Clientes, *front-ends* e gerentes de réplica podem ser Processos separados ou existir no mesmo espaço de endereço.

Muitas vezes, atualizações para essas réplicas podem ser feitas convenientemente por Comunicação em grupo. Expandimos nossa conta do grupo de comunicação para incluir exibições de grupo e comunicação de visualização síncrona.

Definimos linearidade e consistência sequencial como critérios de correção para serviços tolerantes a falhas. Esses critérios expressam como os serviços devem fornecer equivalentes de uma única imagem do conjunto de objetos lógicos, mesmo que esses objetos sejam replicados. O mais praticamente significativo dos critérios é a consistência sequencial.

Na replicação passiva (*backup* primário), a tolerância a falhas é obtida direcionando todas as solicitações por meio de um gerente de réplica diferenciado e com uma réplica de *backup* gerente assumindo se isso falhar.

Na replicação ativa, todos os gerentes de réplica processam todas as solicitações de forma independente, sendo que ambas as formas de replicação podem ser convenientemente implementadas usando Comunicação em Grupo.

Em seguida, consideraremos serviços altamente disponíveis.

- **Fofoca e Bayou:** ambos permitem aos clientes fazerem atualizações em réplicas locais quando particionadas. Em cada Sistema de réplica, os gerentes trocam atualizações uns com os outros quando se reconectam.

Por fim, consideramos o desempenho das transações em relação aos dados replicados: tanto as arquiteturas de *backup* primário, quanto às arquiteturas de *front-ends* podem se comunicar com qualquer gerenciador de réplica existente para esse caso.

Sistemas transacionais permitem falhas de gerenciador de réplica e partições de rede.

# Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

## Leitura

**Controle de concorrência**

<https://goo.gl/n6PvfZ>

**Replicação em sistemas distribuídos**

<https://goo.gl/sWwPY3>

**Arquitetura de sistemas distribuídos**

<https://goo.gl/gTAYTi>



## Referências

GAGLIARDI, Gary. **Cliente/servidor**. São Paulo: Makron Books do Brasil, 1996.

STALLINGS, William. **Arquitetura e organização de computadores**: projeto para o desempenho. 8.ed. São Paulo: Pearson Prentice Hall, 2009.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas Distribuídos**: princípios e paradigmas. 2.ed. Local, Editora, ano.





**Cruzeiro do Sul**  
Educatonal